

EXECUTIVE SUMMARY

Target: http://isolated-2.com
Scan ID: TORTURE-P3-2-ef84c821
Scan Date: 2026-03-04 18:10:57
Scan Duration: N/A
Total Requests Sent: 21
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 2 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 1 | Medium: 1 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk Level: High due to presence of a single high-severity vulnerability
- Most Critical Category: HIGH - requires urgent remediation and mitigation strategies
- Immediate Actions: Prioritize patching or fixing the identified high-risk vulnerability immediately
 - Long-Term Recommendations: Implement comprehensive security monitoring, regular vulnerability assessments

EXECUTIVE STRATEGIC IMPACT

```
{
  "Strategic_Attack_Path_Analysis": [
    "An attacker could exploit the CSRF vulnerability by tricking authenticated users into performing unintended actions on behalf of themselves, potentially granting unauthorized access to sensitive resources.",
    "The SSRF weakness allows an adversary to manipulate internal server requests and retrieve or inject malicious content from within the target network, enabling lateral movement and data exfiltration without detection.",
    "By combining these vulnerabilities in a coordinated attack chain, the threat actor can escalate privileges, bypass security controls, and ultimately achieve full system compromise while remaining undetected through sophisticated obfuscation techniques."
  ]
}
```

DETAILED FINDINGS

FILTER: [ANSWER] AUTHENTICATION GATES

Finding #1: Cross-Site Request Forgery

HIGH

CWE: CWE-352
CVSS Score: 7.5 (HIGH)

THREAT SCORE: 75/100

Description:

- The target endpoint is susceptible to Cross-Site Request Forgery attacks.
- An attacker could trick a user into performing unintended actions on the vulnerable service.
- User authentication and session management are not adequately protected against CSRF.

Impact:

- Data integrity can be compromised, allowing unauthorized modifications or creations of records.
- Confidential information may be exposed through forged requests to sensitive endpoints.
- Malicious transactions could occur without user consent, leading to financial loss or data corruption.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_0
URL: http://test-target-torture-.com/api/v1/endpoint_0
Analysis: The application failed to validate and sanitize user input, allowing malicious requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Cross-Site Request Forgery
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement CSRF tokens in all request forms and ensure they are validated on the server-side before processing.
- Use HTTPOnly cookies for authentication sessions and enforce secure protocols (HTTPS) for all requests.
- Validate and sanitize user input thoroughly, especially when dealing with form submissions or API calls.

Recommended Code Fix:

```
// Example of a CSRF protection mechanism in Node.js using express
const csrf = require('csrf');
app.use(csrf({ cookie: true }));
```

FILTER: SSRF

Finding #2: Server-Side Request Forgery

CRITICAL

CWE: CWE-918
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- The target URL `http://test-target-torture-.com/api/v1/endpoint_1` is vulnerable to Server-Side Request Forgery (SSRF).
- An attacker could potentially manipulate the request payload `'test_payload_1'` to make requests to internal or external services, exposing sensitive information.
- SSRF can lead to unauthorized data access, service abuse, and potential privilege escalation.

Impact:

- Unauthorized data exposure: Attackers may retrieve confidential data from internal systems via forged requests.
- Service disruption: Malicious payloads could trigger unintended responses or cause services to behave unexpectedly.
- Privilege escalation: SSRF can be used to access higher privileged resources than the original request intended.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1
URL: `http://test-target-torture-.com/api/v1/endpoint_1`
Analysis: The application failed to validate and sanitize user input, allowing arbitrary URLs to be passed as parameters.

Table 1: Payload Decomposition

Component	Value	Technical Function
-----------	-------	--------------------

param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category:	Server-Side Request Forgery
Raw Payload:	test_payload_1
Encoded:	746573745f7061796c6f61645f31
Encoding Type:	None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json
{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement strict input validation and sanitization for all user-supplied data before processing requests.
- - Configure firewall rules and network segmentation to restrict outbound connections from web servers.
- - Use a Web Application Firewall (WAF) with SSRF protection capabilities to detect and block suspicious traffic.

Recommended Code Fix:

```
// Example of mitigating SSRF in Node.js using Axios
const axios = require('axios');

function makeRequest(url, payload) {
  return axios.post(url, {payload: JSON.stringify(payload)});
}

// Usage example (ensure proper input validation)
makeRequest('http://test-target-torture-.com/api/v1/endpoint_1', test_payload_1)
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 18:10:57

[Sigma] JOB_ASSIGNED - 2026-03-04 18:10:57

[Gamma] LOG - 2026-03-04 18:10:57

[Kappa] LOG - 2026-03-04 18:10:57

[Beta] LOG - 2026-03-04 18:10:57

[Beta] LOG - 2026-03-04 18:10:57

[Gamma] LOG - 2026-03-04 18:10:58

[Kappa] LOG - 2026-03-04 18:10:58

[Gamma] LOG - 2026-03-04 18:10:58

[Gamma] LOG - 2026-03-04 18:10:58

[Beta] LOG - 2026-03-04 18:10:58

[Beta] LOG - 2026-03-04 18:10:58

[Gamma] LOG - 2026-03-04 18:10:58

[Kappa] LOG - 2026-03-04 18:10:58

[Gamma] LOG - 2026-03-04 18:10:58

[Alpha] LOG - 2026-03-04 18:10:58

[Kappa] LOG - 2026-03-04 18:10:59

[Gamma] VULN_CONFIRMED - 2026-03-04 18:10:59

[Gamma] VULN_CONFIRMED - 2026-03-04 18:10:59

[Gamma] VULN_CONFIRMED - 2026-03-04 18:10:59

[Kappa] GI5_LOG - 2026-03-04 18:11:07